

PASO A PASO

0) Tokenización o análisis léxico, escanea carácter por carácter y devuelve una lista de "tokens" [<SELECT>, <IDENT name>, <, >, <IDENT age>, <FROM>, <IDENT users>, <WHERE>, <IDENT age>, <GT>, <NUMBER 30>, <;>]

1) Parse de la consulta, Análisis sintáctico, Se fija que la secuencia de tokens sea una secuencia válida (por ej. SELECT → lista-de-columnas → FROM → lista-de-tablas → WHERE → condición). Se devuelve un árbol sintáctico que agrupa los token en nodos ej: *SelectStatement*, *ColumnList*, *TableReference*, *WhereClause*, etc.

2) El árbol sintáctico identifica cláusulas y confirma validez de objetos, permisos del usuario y resuelve nombres

3) Forma árbol canónico, acá permite optimización teórica x Álgebra Relacional

Ejemplo de optimizaciones:

1. Empuje de selecciones (Selection push-down)

- Antes:

$$\sigma_{\text{edad} > 30} (\text{Empleados} \bowtie \text{Departamentos})$$

- Después:

$$(\sigma_{\text{edad} > 30} (\text{Empleados})) \bowtie \text{Departamentos}$$

- SQL equivalente:

2. Empuje de proyecciones (Projection push-down)

- Antes:

$$\pi_{\{\text{nombre}, \text{salarior}\}} (\text{Empleados} \bowtie \text{Proyectos})$$

- Después:

$$\pi_{\{\text{nombre}, \text{salarior}\}} \left(\pi_{\{\text{id}, \text{nombre}, \text{salarior}\}} (\text{Empleados}) \bowtie \pi_{\{\text{id}, \text{proyecto}\}} (\text{Proyectos}) \right)$$

4) Empieza a hacer el árbol de ejecución con estadísticas. Acá se generan candidatos de árboles de ejecución y se queda con 1

5) (Executer, Scheduler) Empieza la ejecución por workflow. 1 a 1 (cada "thread serial" porque se crean muchos) ejecuta los read/write de las transacciones.

Más detallado: En el paso 5 ("Executer & Scheduler"), el motor de la base de datos toma el plan físico ya optimizado y lo convierte en una serie de tareas (pipelines) que se asignan a hilos (threads) para su ejecución.

Operaciones que puede hacer el scheduler al recibir una transacción:

Resumen breve

- **Conceder** lock y ejecutar
- **Bloquear/demorar** hasta liberación
- **Abortar** (por deadlock o timeout)

- **Ignorar** lecturas en ciertos isolations
- **Reordenar** operaciones compatibles
- **Escalar/degradar** locks
- **Detectar y resolver deadlocks**
- **Priorizar** transacciones
- **Postergar** escrituras en control optimista

6) (Recovery manager) (Logs) Acá se escriben/leen bloques de queues y cuando se llena el buffer del log lo baja a disco y avisa al scheduler
El recovery manager hace los Aborts y commits

7) (Memory manager/cache manager) Baja efectivamente a disco

ACID:

ATOMICIDAD —> update name y lastName —> desenchufo, si solo uno se actualizo, se rompe

CONSISTENCY —> si no se respetan los check constrains se rompe

ISOLATION —> Read too late/ write too late sin abortar, se rompe

DURABILITY —> Escribo un campo, me dice que esta commiteado, leerlo y no este el cambio.

SEGURIDAD:

INTEGRIDAD —> se rompe cuando alguien no autorizado modifica un dato

CONFIDENCIALIDAD —> se rompe cuando alguien no autorizado accede a un dato

DISPONIBILIDAD —> se rompe cuando alguien autorizado no puede acceder a un dato